

Heverton Eduardo Peres

Memory System: Memória que o Agente Curata & Hiper-Personalização: OMP ao Seu Jeito

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

Memory System: Memória que o Agente Curata	3
O diário de bordo inteligente	3
As Cinco Ferramentas de Memória	3
Os Três Backends de Armazenamento	4
Escopo por Projeto	4
Pipeline de Compressão de Sessões	4
A Metáfora do Estaleiro	5
Configuração Básica do Backend Local	5
Configuração do Backend Mnemopi	5
Uso das Ferramentas de Memória	6
Parâmetros Importantes de Configuração	7
Gerenciamento de Memória via Slash Commands	7
Cena de contraste: o navio sem memória vs. com memória	8
Armadilhas comuns	8
Métricas de eficiência	9
Próximos Passos	9
Hiper-Personalização: OMP ao Seu Jeito	10
Montando o painel de comando completo	10
Config.yml: o painel de instrumentos do navio	10
ModelRoles: 10 papéis, 10 motores	10
Tools: o arsenal sob seu controle	11
Memory: a memória que você cura	11
Configurando o config.yml	12
Configurando o models.yml	12
Magic Keywords: atalhos de poder	14
ACP: integração com editores	14
A Falha na Esteira e a Correção Estrutural	15
Armadilhas comuns	15
Próximos Passos	16

Memory System: Memória que o Agente Curata

O diário de bordo inteligente

No capítulo anterior, você dominou as stream rules — aquelas regras que interceptam o modelo no meio da geração e o colocam de volta nos trilhos. Agora, imagine que cada viagem que você faz pelo código deixa registros no casco do navio: onde ancorou, quais rotas funcionaram, quais equipamentos falharam.

Sem esses registros, a próxima viagem começa do zero. É exatamente isso que o sistema de memória do Oh My Pi resolve: ele permite que o agente recorde decisões técnicas, erros já corrigidos e lições aprendidas entre sessões.

As Cinco Ferramentas de Memória

O Oh My Pi dispõe de cinco ferramentas que trabalham juntas para criar um ciclo completo de conhecimento.

retain — Armazena fatos duradouros no banco de memória ativo. É como registrar no diário de bordo que o porto de Santos tem uma rota específica para descarga de containers.

recall — Busca memórias brutas no banco. Equivale a consultar o arquivo náutico para ver quais rotas já foram testadas com sucesso.

reflect — Sintetiza uma resposta sobre memórias recuperadas. É o capitão que revisa todos os registros anteriores e extrai uma conclusão sobre a melhor rota.

memory_edit — Atualiza, esquece ou invalida memórias armazenadas por ID. Útil quando uma informação se torna desatualizada — como quando uma rota é fechada por obras.

learn — Captura uma lição reutilizável e opcionalmente a promove para uma skill gerenciada. É como criar um manual permanente para a tripulação baseado em experiências reais.

Os Três Backends de Armazenamento

O Oh My Pi permite escolher entre três backends, cada um com características específicas.

local — Armazena resumos e lições gerados a partir de sessões persistidas no projeto. É o diário de bordo que fica guardado no próprio navio.

hindsight — Backend remoto com escopo por banco. Funciona como um arquivo náutico centralizado na base naval, acessível por diferentes navios.

mnemopi — Backend local baseado em SQLite. Oferece recall polifônico (vetorial, grafos, fatos, temporal) e é o mais completo para uso individual.

Escopo por Projeto

Cada backend suporta diferentes modos de escopo.

global — Um banco compartilhado para todos os projetos. É como um arquivo náutico central que todos os navios da frota consultam.

per-project — Memória isolada por projeto. Cada navio tem seu próprio diário de bordo privado.

per-project-tagged — Escrita local com visibilidade global. O navio registra em seu diário, mas pode consultar o arquivo central da frota.

Pipeline de Compressão de Sessões

O sistema local implementa um pipeline de duas fases para consolidar o conhecimento.

Fase 1 — Extração por sessão: Para cada sessão passada, um modelo lê o histórico e extrai sinal duradouro: decisões técnicas, restrições, falhas resolvidas e fluxos de trabalho recorrentes.

Fase 2 — Consolidação: Um segundo modelo lê todas as extrações e produz três saídas: `MEMORY.md` (documento de memória de longo prazo curado), `memory_summary.md` (texto compacto injetado no início da sessão), e `skills/` (playbooks procedimentais reutilizáveis).

A Metáfora do Estaleiro

Imagine que você é o mestre de um estaleiro digital. Cada navio que sai para o mar coleta dados sobre as condições do oceano, portos visitados e equipamentos utilizados. Quando o navio retorna ao estaleiro, os dados são processados.

`retain` é como anotar no diário de bordo: “O porto de Santos exige autorização prévia para containers de 40 pés”. `recall` é consultar o arquivo de rotas anteriores quando o navio precisa ir a Santos novamente. `reflect` é o capitão revisando todos os registros e decidindo: “Baseado nas últimas 5 viagens, a melhor rota para Santos passa por Angra dos Reis”. `memory_edit` é atualizar o diário quando uma rota é fechada. `learn` é criar um manual permanente: “Procedimento padrão para descarga em portos com maré alta”.

Configuração Básica do Backend Local

Para ativar o backend de memória local, adicione ao seu `config.yml`.

```
memory:
  backend: local

autolearn:
  enabled: true
```

Com essa configuração, o Oh My Pi ativará o pipeline de memória que gera resumos e lições entre sessões. O backend local é ideal para quem está começando, pois não requer configuração de servidores externos.

Configuração do Backend Mnemopi

Para usar o Mnemopi, que oferece funcionalidades mais avançadas como `recall` polifônico.

```
memory:
  backend: mnemopi

mnemopi:
  scoping: per-project-tagged
  autoRecall: true
  autoRetain: true
```

```
polyphonicRecall: true  
retainEveryNTurns: 4  
recallLimit: 8
```

Uso das Ferramentas de Memória

Durante uma sessão, o agente pode usar as ferramentas de memória.

Retain — Armazenar um fato importante:

```
retain(  
  content="O projeto usa Node.js v20 LTS com npm como gerenciador de  
pacotes",  
  tags=["configuracao", "nodejs", "projeto"]  
)
```

Recall — Buscar memórias relevantes:

```
recall(  
  query="configuração do projeto e dependências",  
  limit=5  
)
```

Reflect — Sintetizar informações:

```
reflect(  
  query="quais são as melhores práticas de configuração para este  
projeto?"  
)
```

Memory_edit — Atualizar memória desatualizada:

```
memory_edit(  
  id="mem_123",  
  action="update",  
  content="O projeto migrou de npm para pnpm v9"  
)
```

Learn — Capturar uma lição reutilizável:

```
learn(  
  content="Sempre usar pnpm em projetos com workspaces para evitar  
conflitos de dependências",  
)
```

```
context="Descoberto após problemas com npm em monorepo com 15 pacotes"
)
```

Parâmetros Importantes de Configuração

Parâmetro	Padrão	Descrição
memories.maxRolloutAgeDays	30	Sessões mais antigas não são processadas
memories.minRolloutIdleHours	12	Sessões ativas recentemente são ignoradas
memories.maxRolloutsPerStartup	64	Limite de sessões processadas por inicialização
memories.summaryInjectionTokenLimit	5000	Limite de tokens para injeção de resumo

Para o Mnemopi, parâmetros adicionais.

Parâmetro	Padrão	Descrição
mnemopi.polyphonicRecall	false	Ativa recall em 4 vozes
mnemopi.retainEveryNTurns	4	Número mínimo de turns entre retentions automáticas
mnemopi.recallLimit	8	Máximo de memórias recuperadas no prompt

Gerenciamento de Memória via Slash Commands

```
# Ver a injeção de memória atual
/memory view

# Ver estatísticas do backend
/memory stats
```

```
# Diagnosticar problemas  
/memory diagnose
```

```
# Limpar dados de memória  
/memory clear
```

```
# Forçar consolidação  
/memory enqueue
```

Cena de contraste: o navio sem memória vs. com memória

Você está trabalhando em um projeto de API REST com FastAPI. Na segunda-feira, descobriu que o banco de dados precisa de uma configuração específica de connection pooling para suportar 100 requisições simultâneas. Você documentou isso em um arquivo `NOTAS.md`.

Na terça-feira, uma nova sessão começa. O agente de IA não tem contexto sobre a descoberta de segunda-feira. Ele sugere uma configuração padrão que causa timeout em produção. Você perde 30 minutos debugando o problema.

Agora, imagine que o sistema de memória estava ativo. Na segunda-feira, quando você descobriu a configuração correta, o agente usou `retain` para armazenar: “FastAPI com SQLAlchemy precisa de `pool_size=20` e `max_overflow=10` para 100 requisições simultâneas”. Na terça-feira, na primeira interação, o agente usou `recall` e encontrou essa memória. Ele aplicou automaticamente a configuração correta, evitando o problema.

Armadilhas comuns

Backend não configurado. Muitos iniciantes esquecem de ativar o backend. Sem `memory.backend: local`, as ferramentas de memória não ficam disponíveis.

Limite de tokens excedido. O resumo injetado no início da sessão tem limite de 5000 tokens. Se a memória acumulada for grande demais, informações importantes podem ser truncadas.

Memória desatualizada. Se o código muda drasticamente entre sessões, a memória pode conter informações obsoletas. Use `memory_edit` para atualizar ou invalidar memórias antigas.

Escopo inadequado. Usar `global` quando deveria ser `per-project` pode vazar informações sensíveis de um projeto para outro.

Métricas de eficiência

Com o sistema de memória ativo, você pode esperar redução de tempo de setup de ~5 minutos (re-explicar contexto) para ~0 segundos (injeção automática), consistência de configurações com 100% das configurações importantes preservadas entre sessões, e velocidade de onboarding com novos membros da equipe herdando o conhecimento acumulado automaticamente.

Próximos Passos

Neste capítulo, você explorou o sistema de memória do Oh My Pi — a âncora que mantém o conhecimento do agente firme entre sessões. Cinco ferramentas complementares — retain, recall, reflect, memory_edit e learn criam um ciclo completo de captura, busca e síntese de conhecimento. Três backends flexíveis — local para uso individual, Hindsight para equipes, e Mnemopi para funcionalidades avançadas. E um pipeline inteligente de compressão que extrai e consolida automaticamente o conhecimento mais relevante das sessões passadas.

Experimente ativar o backend local em um projeto real. Comece com `memory.backend: local` e `autolearn.enabled: true`. Após algumas sessões, use `/memory view` para ver o que o agente aprendeu.

No próximo capítulo, você descobrirá como o Oh My Pi integra editores de código diretamente no agente — a ferramenta ACP que permite ao modelo ler e escrever no buffer que você está visualizando.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.

Hiper-Personalização: OMP ao Seu Jeito

Montando o painel de comando completo

No capítulo anterior, você configurou o sistema de memória do OMP — aquela âncora que mantém o agente lembrando de fatos, lições e preferências entre sessões. Mas a memória é apenas uma peça do quebra-cabeça.

O verdadeiro poder do OMP aparece quando você personaliza cada aspecto do harness: escolhe qual modelo roda em cada papel, quais ferramentas ficam habilitadas, quais regras guiam o comportamento e como tudo isso se integra ao editor que você já usa todos os dias.

Config.yml: o painel de instrumentos do navio

Todo estaleiro funcional tem um painel central — um lugar onde o mestre ajusta motor, leme, instrumentos e comunicações. No OMP, esse painel é o arquivo `~/.omp/agent/config.yml`. Nele, você define três coisas fundamentais: quais modelos rodam em cada papel (`modelRoles`), quais ferramentas estão habilitadas (`tools`) e como a memória persiste entre sessões (`memory`).

O `config.yml` é lido quando o OMP inicia. Qualquer alteração nele exige reiniciar a sessão para ter efeito. Pense nele como o manual de operações do seu navio — ajustar o leme em alto-mar é possível, mas o ideal é calibrar tudo antes de zarpar.

ModelRoles: 10 papéis, 10 motores

O OMP não trata todos os turnos da mesma forma. Ele diferencia 10 papéis distintos, cada um com suas necessidades de velocidade, raciocínio e custo.

Role	Uso	Característica
<code>default</code>	Turnos normais	Equilíbrio entre custo e qualidade
<code>smol</code>	Fan-out de subagentes	Modelo leve e barato para tarefas paralelas
<code>slow</code>	Raciocínio profundo	Modelo potente para problemas complexos
<code>plan</code>	Modo plano	Focado em planejamento, não execução
<code>commit</code>	Changelogs	Geração de mensagens de commit
<code>vision</code>	Análise de imagens	Processamento visual
<code>designer</code>	Design de interfaces	Geração de layouts
<code>task</code>	Orquestração	Coordenação de tarefas
<code>advisor</code>	Revisão inline	Segundo olho em cada turno
<code>tiny</code>	Tarefas leves	Rápido e econômico para operações simples

A mágica está em mapear cada papel ao modelo certo. Você pode usar um modelo potente e caro para `slow` (problemas difíceis) e um modelo leve e barato para `smol` (dezenas de subagentes trabalhando em paralelo). Isso reduz custos sem sacrificar qualidade onde ela importa.

Tools: o arsenal sob seu controle

O OMP vem com 31 ferramentas built-in, mas nem todas precisam estar ativas o tempo todo. O campo `tools` no `config.yml` permite habilitar ou desabilitar ferramentas específicas. Se você não trabalha com browser automation, pode desligar a ferramenta `browser` e reduzir o consumo de tokens por turno. Se seu projeto não usa debug nativo, desative `debug`.

Memory: a memória que você cura

No Capítulo 15, você viu como o sistema de memória funciona. Agora, no `config.yml`, você escolhe o backend (local, Hindsight ou Mnemopi) e o escopo (projeto ou global). Essa escolha afeta onde os dados persistem e com quem são compartilhados.

Configurando o config.yml

Vamos montar um config.yml completo.

```
# ~/.omp/agent/config.yml

modelRoles:
  default: anthropic/claude-sonnet-4-20250514
  slow: anthropic/claude-opus-4-0
  smol: openai/gpt-4o-mini
  advisor: anthropic/claude-sonnet-4-20250514
  plan: anthropic/claude-sonnet-4-20250514

tools:
  enabled:
    - read
    - write
    - edit
    - bash
    - grep
    - glob
    - lsp
    - debug
    - task
    - browser
  disabled:
    - security_scan
    - generate_image

memory:
  backend: local
  scope: project
```

Configurando o models.yml

Agora o models.yml — a oficina de motores. Aqui definimos providers customizados, incluindo modelos locais.

```
# ~/.omp/agent/models.yml

providers:
  spark:
```

```
baseUrl: http://192.168.10.223:8000/v1
api: openai-completions
apiKey: dummy
models:
  - id: minimax-m3
    name: MiniMax M3
    contextWindow: 100000
    maxTokens: 32000

anthropic:
  api: anthropic
  apiKey: <sua-chave-anthropic>
  models:
    - id: claude-sonnet-4-20250514
      name: Claude Sonnet 4
      contextWindow: 200000
      maxTokens: 8192
    - id: claude-opus-4-0
      name: Claude Opus 4
      contextWindow: 200000
      maxTokens: 32768

openai:
  api: openai
  apiKey: <sua-chave-openai>
  models:
    - id: gpt-4o-mini
      name: GPT-4o Mini
      contextWindow: 128000
      maxTokens: 16384

modelRoles:
  default: spark/minimax-m3
  smol: openai/gpt-4o-mini
  slow: anthropic/claude-opus-4-0
  plan: anthropic/claude-sonnet-4-20250514
  advisor: anthropic/claude-sonnet-4-20250514
```

Observe que o `modelRoles` pode aparecer tanto no `config.yml` quanto no `models.yml`. Quando presente nos dois, o `models.yml` tem prioridade — é ele que define o mapeamento final entre provider e papel.

Magic Keywords: atalhos de poder

O OMP reconhece três palavras mágicas que você pode incluir em qualquer mensagem para alterar o comportamento do agente. Essas keywords são processadas pelo harness antes de enviar ao modelo.

Keyword	Efeito	Quando usar
<code>ultrathink</code>	Raciocínio multi-step cuidadoso	Problemas complexos que exigem análise profunda
<code>orchestrate</code>	Fan-out paralelo com verificação	Tarefas que se beneficiam de múltiplos workers
<code>workflowz</code>	Workflow determinístico multi-subagent	Pipelines com etapas bem definidas

Exemplo de uso:

> `ultrathink` Analise a arquitetura deste módulo e proponha melhorias de performance

> `orchestrate` Refatore os 10 arquivos de teste em paralelo, garantindo que cada um passe no lint

> `workflowz` 1. Extraia dados do CSV 2. Valide schema 3. Gere relatório 4. Compile PDF

Cada keyword desencadeia um modo de operação diferente no harness. O `ultrathink` faz o agente pausar e pensar antes de agir. O `orchestrate` distribui trabalho entre subagentes. O `workflowz` cria uma pipeline determinística onde cada etapa alimenta a próxima.

ACP: integração com editores

O ACP (Agent Control Protocol) é o que permite rodar o OMP dentro de editores como Zed. Em vez de alternar entre terminal e editor, você mantém o agente integrado ao seu ambiente de trabalho.

```
acp:
  enabled: true
```

```
editor: zed
save_path: /tmp/omp-acp-output
```

Quando o ACP está ativo, o OMP lê o buffer atual do editor, processa a instrução e escreve o resultado de volta. O fluxo é: você seleciona código no editor, envia um comando via ACP, o OMP lê o buffer, processa e gera a resposta, o resultado é escrito no `save_path`, e o editor atualiza o buffer.

A Falha na Esteira e a Correção Estrutural

Você está trabalhando em um projeto grande com 15 módulos. Abre o terminal e inicia o OMP com a configuração padrão. O agente começa a trabalhar, mas algo está errado: ele está usando o modelo mais caro para tarefas simples de rename, e modelos baratos para problemas de arquitetura que exigem raciocínio profundo. O custo de tokens dispara e a qualidade cai nos pontos que mais importam.

O problema é que você não configurou as `modelRoles`. O OMP estava usando o mesmo modelo para tudo — como um navio que navega em velocidade máxima mesmo em porto, gastando combustível à toa.

A correção: você abre o `models.yml` e mapeia cada papel ao modelo certo.

```
modelRoles:
  default: spark/minimax-m3      # Tarefas normais – barato e rápido
  slow: anthropic/claude-opus-4-0 # Problemas difíceis – potente
  smol: openai/gpt-4o-mini       # Subagentes – o mais econômico
```

Agora, quando o agente precisa de raciocínio profundo, ele sobe para o Opus. Quando distribui trabalho entre subagentes, usa o Mini. O custo cai significativamente sem sacrificar qualidade onde ela realmente importa.

Armadilhas comuns

Esquecer de reiniciar a sessão após editar `config.yml`. As alterações só têm efeito na próxima inicialização do OMP.

Mapear todos os papéis ao mesmo modelo. Isso anula a vantagem de ter 10 roles distintos — use modelos diferentes para papéis diferentes.

Habilitar ferramentas desnecessárias. Cada ferramenta adiciona tokens ao system prompt. Se você não usa `browser`, desative-o.

Não usar magic keywords. São gratuitas e podem transformar a qualidade do resultado em tarefas complexas. O `ultrathink` sozinho evita erros que o agente cometeria em modo padrão.

Configurar memory scope como `global` sem necessidade. Dados globais vaziam o contexto em todos os projetos. Use `project` por padrão.

Próximos Passos

Neste capítulo, você montou o painel de comando completo do seu estaleiro digital. Os três pilares — `config.yml` (instrumentos), `models.yml` (motores) e ACP + magic keywords (integração e atalhos) — transformam o OMP de ferramenta genérica em uma extensão personalizada do seu fluxo de trabalho.

Escolheu qual modelo roda em cada papel, configurou quais ferramentas ficam habilitadas, definiu o backend de memória e integrou o agente ao seu editor. Essa é a diferença entre usar o OMP e comandá-lo — e é exatamente o que separa um iniciante de um Mestre de Estaleiro Digital.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi. Junte-se a mais de 23.3k desenvolvedores que já estão usando o harness mais completo do mercado.
